Proof Production for Satisfiability Modulo Finite Fields with Proof Checking in Pacheck and Lean

Pedro Saccomani[1] Abdalrhman Mohamed[2] Elizaveta Pertseva[2]
Daniela Kaufmann[3] Cesare Tinelli[4] Clark Barrett[2] Haniel Barbosa[1]

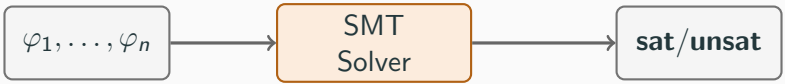
[1]Universidade Federal de Minas Gerais [2]Stanford University
[3]TU Wien [4]The University of Iowa

FMCAD 2026

Introduction

Satisfiability Modulo Theories

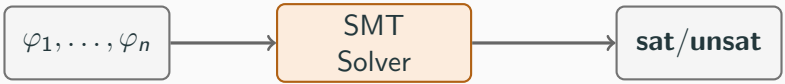
SMT solvers decide the satisfiability of first-order formulas modulo background theories: arithmetic, bit-vectors, arrays, strings, ..., finite fields.



- The backbone of automated verification: model checkers, program verifiers (Dafny, Why3), symbolic execution engines...;
- For verification, we are usually concerned with **unsat**: the negation of a goal has no counterexample, that is, the goal holds;
- Finite fields: application to the verification of cryptographic systems, e.g. zero knowledge proof compilers [**Ozdemir et al.**, **2023**].

Satisfiability Modulo Theories

SMT solvers decide the satisfiability of first-order formulas modulo background theories: arithmetic, bit-vectors, arrays, strings, ..., finite fields.



- The backbone of automated verification: model checkers, program verifiers (Dafny, Why3), symbolic execution engines...;
- For verification, we are usually concerned with **unsat**: the negation of a goal has no counterexample, that is, the goal holds;
- Finite fields: application to the verification of cryptographic systems, e.g. zero knowledge proof compilers [**Ozdemir et al.**, **2023**].

Why should we trust the output of a solver?

SMT Proofs

SMT solvers can be hard to trust:

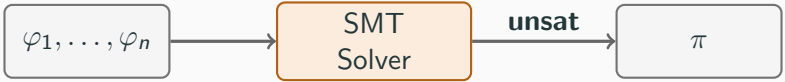
- Code bases are large and complex ($\sim$300k LOC in cvc5);
- Despite the effort of developers, bugs can appear...

SMT Proofs

SMT solvers can be hard to trust:

- Code bases are large and complex (∼300k LOC in cvc5);
- Despite the effort of developers, bugs can appear...

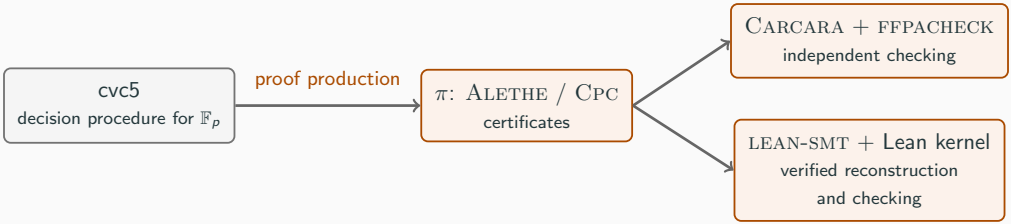
Solution: certifying solvers.



- π is a *proof certificate*: checked *independently* by a small, efficient checker;
- cvc5 [**Barbosa et al.**, **2022**] emits certificates in two formats: ALETHE [**Schurr et al.**, **2021**] and CPC [**Reynolds et al.**, **2026**];
- Certificates can also be *replayed* inside proof assistants: LEAN-SMT in Lean and `smt` in Isabelle.

Our Goals

Make the SMT theory of finite fields proof producing, with checking support:



Background - Ideals and Varieties

Fix a prime p and variables $\mathbf{X}$; $\mathbb{F}_p[\mathbf{X}]$ is the set of polynomials over $\mathbb{F}_p$.

Definition (Ideal and variety of $G = \{g_1, \ldots, g_k\} \subseteq \mathbb{F}_p[\mathbf{X}]$)

$$\langle G \rangle = \left\{ \sum_{i=1}^{k} h_i \cdot g_i \;\middle|\; h_i \in \mathbb{F}_p[\mathbf{X}] \right\} \qquad \mathcal{V}(\langle G \rangle) = \{\mathbf{x} \in \mathbb{F}_p^n \mid f(\mathbf{x}) = 0 \text{ for all } f \in \langle G \rangle\}$$

$\mathcal{V}(\langle G \rangle)$ is exactly the set of common roots of G.

Background - Ideals and Varieties

Fix a prime p and variables $\mathbf{X}$; $\mathbb{F}_p[\mathbf{X}]$ is the set of polynomials over $\mathbb{F}_p$.

Definition (Ideal and variety of $G = \{g_1, \ldots, g_k\} \subseteq \mathbb{F}_p[\mathbf{X}]$)

$$\langle G \rangle = \left\{ \sum_{i=1}^{k} h_i \cdot g_i \;\middle|\; h_i \in \mathbb{F}_p[\mathbf{X}] \right\} \qquad \mathcal{V}(\langle G \rangle) = \{\mathbf{x} \in \mathbb{F}_p^n \mid f(\mathbf{x}) = 0 \text{ for all } f \in \langle G \rangle\}$$

$\mathcal{V}(\langle G \rangle)$ is exactly the set of common roots of G.

- If $1 \in \langle G \rangle$, then $\mathcal{V}(\langle G \rangle) = \emptyset$: the decision procedure main tool for concluding unsatisfiability;

Background - Ideals and Varieties

Fix a prime p and variables $\mathbf{X}$; $\mathbb{F}_p[\mathbf{X}]$ is the set of polynomials over $\mathbb{F}_p$.

Definition (Ideal and variety of $G = \{g_1, \ldots, g_k\} \subseteq \mathbb{F}_p[\mathbf{X}]$)

$$\langle G \rangle = \left\{ \sum_{i=1}^{k} h_i \cdot g_i \;\middle|\; h_i \in \mathbb{F}_p[\mathbf{X}] \right\} \qquad \mathcal{V}(\langle G \rangle) = \{\mathbf{x} \in \mathbb{F}_p^n \mid f(\mathbf{x}) = 0 \text{ for all } f \in \langle G \rangle\}$$

$\mathcal{V}(\langle G \rangle)$ is exactly the set of common roots of G.

- If $1 \in \langle G \rangle$, then $\mathcal{V}(\langle G \rangle) = \emptyset$: the decision procedure main tool for concluding unsatisfiability;
- Unfortunately, the converse does not hold over $\mathbb{F}_p$: e.g., $\mathcal{V}(\langle x^3 + 2x + 2 \rangle) = \emptyset$ in $\mathbb{F}_3$, but $1 \notin \langle x^3 + 2x + 2 \rangle$.

Proof Calculus

Decision Procedure - Overview

In: A set of $\mathbb{F}_p$-literals L in variables X [**Ozdemir et al.**, **2023**]
Out: UNSAT/SAT
DECISION PROCEDURE
$G \leftarrow \emptyset$; $w_i \leftarrow \text{freshVar}, \forall i$;
for $(s_i \bowtie_i t_i \in L)$
  $s_i \leftarrow \mathit{Rewrite}(s_i)$; $t_i \leftarrow \mathit{Rewrite}(t_i)$;
  if $(\bowtie_i = \approx)$ $G \leftarrow G \cup \{s_i - t_i\}$;
  elif $(\bowtie_i = \not\approx)$ $G \leftarrow G \cup \{w_i(s_i - t_i) - 1\}$;
$\mathbf{B} \leftarrow \mathbf{GB}(\mathbf{G})$;
if $(1 \Rightarrow_B 0)$ **return** UNSAT;
$m \leftarrow \mathbf{FindZero}(\mathbf{B})$;
if $(m = \bot)$ **return** UNSAT;
else return SAT;

Our proofs must capture four kinds of reasoning: **term rewriting**, **literal → polynomial translation**, **ideal membership**, and **root search**.

Proof Calculus - Justifying Polynomial Equalities

We capture each step of the run with **inference rules**. First, rewriting: term rewriting

$$x(x-1)(x-2) \approx 1 \quad \rightsquigarrow \quad x^3 + 2x + 2 \approx 0 \qquad (\text{over } \mathbb{F}_3)$$

Proof Calculus - Justifying Polynomial Equalities

We capture each step of the run with **inference rules**. First, rewriting: term rewriting

$$x(x-1)(x-2) \approx 1 \quad \rightsquigarrow \quad x^3 + 2x + 2 \approx 0 \qquad \text{(over } \mathbb{F}_3\text{)}$$

- Terms with the same arithmetic normal form are equal (`ff_poly_norm`):

$$\frac{- \quad | \quad s, t}{t \approx s} \text{ if } \mathtt{Norm}(t) \approx \mathtt{Norm}(s)$$

(premises | arguments above the line, conclusion below, optional side condition)

Proof Calculus - Justifying Polynomial Equalities

We capture each step of the run with **inference rules**. First, rewriting: term rewriting

$$x(x-1)(x-2) \approx 1 \quad \rightsquigarrow \quad x^3 + 2x + 2 \approx 0 \qquad \text{(over } \mathbb{F}_3\text{)}$$

- Terms with the same arithmetic normal form are equal (`ff_poly_norm`):

$$\frac{- \quad | \quad s, t}{t \approx s} \text{ if } \mathtt{Norm}(t) \approx \mathtt{Norm}(s)$$

(premises | arguments above the line, conclusion below, optional side condition)

- It proves the differences equal: $1 \cdot (x(x-1)(x-2) - 1) \approx 1 \cdot (x^3 + 2x + 2)$;
- `ff_poly_norm_eq` lifts this to the literals:
  $(x(x-1)(x-2) \approx 1) \approx (x^3 + 2x + 2 \approx 0)$.

Proof Calculus - Problem Framing Conversion

Next, the run frames the literals as one polynomial system: translation

$$\{x^2 \approx x,\ x \not\approx 0,\ x \not\approx 1\} \ \rightsquigarrow\ \mathcal{V}(\langle x^2 + 2x,\ w_1 x - 1,\ w_2(x-1) - 1\rangle) \not\approx \emptyset$$

Proof Calculus - Problem Framing Conversion

Next, the run frames the literals as one polynomial system: translation

$$\{x^2 \approx x,\ x \not\approx 0,\ x \not\approx 1\} \ \rightsquigarrow\ \mathcal{V}(\langle x^2 + 2x,\ w_1 x - 1,\ w_2(x-1) - 1\rangle) \not\approx \emptyset$$

- Equalities normalize: $x^2 \approx x \ \rightsquigarrow\ x^2 + 2x \approx 0$, by `ff_poly_norm(_eq)`;

Proof Calculus - Problem Framing Conversion

Next, the run frames the literals as one polynomial system: translation

$$\{x^2 \approx x,\ x \not\approx 0,\ x \not\approx 1\} \ \rightsquigarrow\ \mathcal{V}(\langle x^2 + 2x,\ w_1 x - 1,\ w_2(x-1) - 1\rangle) \not\approx \emptyset$$

- Equalities normalize: $x^2 \approx x \ \rightsquigarrow\ x^2 + 2x \approx 0$, by `ff_poly_norm(_eq)`;
- Disequalities hold iff the difference has an *inverse*: $x \not\approx 0 \ \rightsquigarrow\ w_1 \cdot x - 1 \approx 0$, introduced by a specialized rule, `ff_diseq`;

Proof Calculus - Problem Framing Conversion

Next, the run frames the literals as one polynomial system: translation

$$\{x^2 \approx x,\ x \not\approx 0,\ x \not\approx 1\} \ \rightsquigarrow\ \mathcal{V}(\langle x^2 + 2x,\ w_1 x - 1,\ w_2(x-1) - 1\rangle) \not\approx \emptyset$$

- Equalities normalize: $x^2 \approx x \ \rightsquigarrow\ x^2 + 2x \approx 0$, by `ff_poly_norm(_eq)`;
- Disequalities hold iff the difference has an *inverse*: $x \not\approx 0 \ \rightsquigarrow\ w_1 \cdot x - 1 \approx 0$, introduced by a specialized rule, `ff_diseq`;
- The last step, `ff_poly_conversion`, assembles the system: a solution is a common root.

Proof Calculus - Deciding Ideal Membership

When the algorithm run finds $1 \in \langle G \rangle$ (no common roots), the proof records only the linear combination:

ideal membership

$$\begin{aligned} 1 &= w_1 w_2 \cdot (x^2 + 2x) \\ &+ w_2(1 - x) \cdot (w_1 x - 1) \\ &+ 2 \cdot (w_2(x - 1) - 1) \end{aligned}$$

Proof Calculus - Deciding Ideal Membership

When the algorithm run finds $1 \in \langle G \rangle$ (no common roots), the proof records only the linear combination:

ideal membership

$$\begin{aligned} 1 &= w_1 w_2 \cdot (x^2 + 2x) \\ &+ w_2(1 - x) \cdot (w_1 x - 1) \\ &+ 2 \cdot (w_2(x - 1) - 1) \end{aligned}$$

- Each generator's membership in $\langle G \rangle$ is an axiom (`ff_ideal_generator`);
- `ff_poly_combination` sums them with the multipliers from the run:

$$\frac{r_1 \in \langle G \rangle, \ \ldots, \ r_k \in \langle G \rangle \quad | \quad m_1, \ldots, m_k}{\sum_{i=1}^{k} m_i \cdot r_i \in \langle G \rangle}$$

Proof Calculus - Deciding Ideal Membership

When the algorithm run finds $1 \in \langle G \rangle$ (no common roots), the proof records only the linear combination:

ideal membership

$$\begin{aligned} 1 &= w_1 w_2 \cdot (x^2 + 2x) \\ &+ w_2(1 - x) \cdot (w_1 x - 1) \\ &+ 2 \cdot (w_2(x - 1) - 1) \end{aligned}$$

- Each generator's membership in $\langle G \rangle$ is an axiom (`ff_ideal_generator`);
- `ff_poly_combination` sums them with the multipliers from the run:

$$\frac{r_1 \in \langle G \rangle, \ \ldots, \ r_k \in \langle G \rangle \quad | \quad m_1, \ldots, m_k}{\sum_{i=1}^{k} m_i \cdot r_i \in \langle G \rangle}$$

- The *unsat conclusion* rule turns $1 \in \langle G \rangle$ into $\mathcal{V}(\langle G \rangle) \approx \emptyset$: contradiction.

Proof Calculus - Searching for Solutions

For the cubic, membership is inconclusive ($1 \notin \langle B \rangle$), yet

root search

$$\texttt{Roots}(x^3 + 2x + 2) = \emptyset \text{ in } \mathbb{F}_3$$

Proof Calculus - Searching for Solutions

For the cubic, membership is inconclusive ($1 \notin \langle B \rangle$), yet

root search

$$\texttt{Roots}(x^3 + 2x + 2) = \emptyset \text{ in } \mathbb{F}_3$$

- The procedure falls back to root search: branch on candidate values for a variable. If the search fails, the proof must justify why no value works:

Proof Calculus - Searching for Solutions

For the cubic, membership is inconclusive ($1 \notin \langle B \rangle$), yet

root search

$$\texttt{Roots}(x^3 + 2x + 2) = \emptyset \text{ in } \mathbb{F}_3$$

- The procedure falls back to root search: branch on candidate values for a variable. If the search fails, the proof must justify why no value works:
- Common roots of G lie among the roots of any *univariate* $f \in \langle G \rangle$ (`ff_root_branch`):

$$\frac{\mathcal{V}(\langle G \rangle) \not\approx \emptyset, \quad f \in \langle G \rangle \quad | \quad x, \texttt{Roots}(f)}{\bigvee_{v \in \texttt{Roots}(f)} \mathcal{V}(\langle G \cup \{x - v\} \rangle) \not\approx \emptyset}$$

Proof Calculus - Searching for Solutions

For the cubic, membership is inconclusive ($1 \notin \langle B \rangle$), yet

root search

$$\texttt{Roots}(x^3 + 2x + 2) = \emptyset \text{ in } \mathbb{F}_3$$

- The procedure falls back to root search: branch on candidate values for a variable. If the search fails, the proof must justify why no value works:
- Common roots of G lie among the roots of any *univariate* $f \in \langle G \rangle$ (`ff_root_branch`):

$$\frac{\mathcal{V}(\langle G \rangle) \not\approx \emptyset, \quad f \in \langle G \rangle \quad \mid \quad x, \texttt{Roots}(f)}{\bigvee_{v \in \texttt{Roots}(f)} \mathcal{V}(\langle G \cup \{x - v\} \rangle) \not\approx \emptyset}$$

- One application refutes the cubic: no roots, so the empty disjunction, i.e., $\bot$;
- With no univariate polynomial, a specialized rule, `ff_exhaust_branch`, branches on every value of $\mathbb{F}_p$ for a single variable.

Proof Production - Implementation

We instrumented cvc5 (leveraging its proof infrastructure) and its dependency CoCoALib [**Abbott & Bigatti**], which performs the Gröbner basis computation and ideal membership testing:

- **New operators**: `@ff.ideal` and `@ff.variety`, representing the ideal of a set of polynomials and its variety;

Proof Production - Implementation

We instrumented cvc5 (leveraging its proof infrastructure) and its dependency CoCoALib [**Abbott & Bigatti**], which performs the Gröbner basis computation and ideal membership testing:

- **New operators**: `@ff.ideal` and `@ff.variety`, representing the ideal of a set of polynomials and its variety;
- **Instrumenting CoCoALib**: premises are captured via high-level hooks (at the level of algorithms), *multipliers* must be intercepted at a lower level (at level of data structures).

Proof Production - Implementation

We instrumented cvc5 (leveraging its proof infrastructure) and its dependency CoCoALib [**Abbott & Bigatti**], which performs the Gröbner basis computation and ideal membership testing:

- **New operators**: `@ff.ideal` and `@ff.variety`, representing the ideal of a set of polynomials and its variety;
- **Instrumenting CoCoALib**: premises are captured via high-level hooks (at the level of algorithms), *multipliers* must be intercepted at a lower level (at level of data structures).
- **Two representations**: an augmented interface translates between cvc5 terms and CoCoALib polynomials.

Proof Checking

Proof Checking - Pipeline



- CARCARA [**Andreotti et al.**, **2023**] (extended with finite field sorts and terms) checks preprocessing, Boolean reasoning, and the conversion and normalization rules (`ff_poly_conversion`, `ff_poly_norm(_eq)`, `ff_diseq`);

Proof Checking - Pipeline



- CARCARA [**Andreotti et al.**, **2023**] (extended with finite field sorts and terms) checks preprocessing, Boolean reasoning, and the conversion and normalization rules (`ff_poly_conversion`, `ff_poly_norm(_eq)`, `ff_diseq`);
- All remaining theory reasoning is packaged into one ALETHE rule, discharged by invoking FFPACHECK:

$$\frac{\mathcal{V}(\langle G \rangle) \not\approx \emptyset \quad | \quad \texttt{pac_proof}}{\bot}$$

Proof Checking - Pipeline



- CARCARA [**Andreotti et al.**, **2023**] (extended with finite field sorts and terms) checks preprocessing, Boolean reasoning, and the conversion and normalization rules (`ff_poly_conversion`, `ff_poly_norm(_eq)`, `ff_diseq`);
- All remaining theory reasoning is packaged into one ALETHE rule, discharged by invoking FFPACHECK:

$$\frac{\mathcal{V}(\langle G \rangle) \not\approx \emptyset \quad | \quad \texttt{pac_proof}}{\bot}$$

- The CPC format instead mirrors the calculus rule by rule; that is the format LEAN-SMT reconstructs.

Extending PAC and Pacheck - Finite Field Terms

PAC: a proof format for algebraic reasoning, checked efficiently by PACHECK [**Kaufmann et al.**, **2024**]:

- Proofs consist of axioms (generators) plus linear combination steps;
- Invariant: every derived polynomial lies in the ideal of the axioms;
- Originally defined over pseudo-Boolean polynomials ($x \cdot x \to x$, integer coefficients).

Extending PAC and Pacheck - Finite Field Terms

PAC: a proof format for algebraic reasoning, checked efficiently by PACHECK [**Kaufmann et al.**, **2024**]:

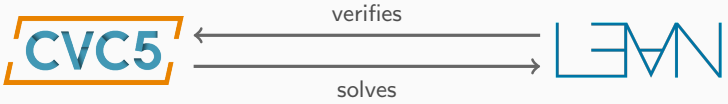
- Proofs consist of axioms (generators) plus linear combination steps;
- Invariant: every derived polynomial lies in the ideal of the axioms;
- Originally defined over pseudo-Boolean polynomials ($x \cdot x \to x$, integer coefficients).

Our lift to $\mathbb{F}_p$, FFPACHECK:

- The `m p;` rule fixes the modulus up front;
- Coefficients are reduced mod p after every operation;
- Functional equivalence over $\mathbb{F}_p$: polynomials simplified under $x^p \to x$;
- New branching steps handle the root search procedure.

Proof Reconstruction and Automation

lean-smt



LEAN-SMT [**Mohamed et al.**, **2025**] discharges Lean goals via SMT, replaying the proof through Lean's kernel. We extended it with the finite field calculus:

```
example [Fact (Nat.Prime 3)] (x : ZMod 3) : ¬(x * (x - 1) * (x - 2) = 1) := by smt
```

1. The goal over `ZMod 3` is negated and translated to an SMT query over `(_ FiniteField 3)`;
2. cvc5 answers **unsat** with a CPC proof;
3. Each calculus rule application is discharged by a corresponding Lean lemma.

Representing Polynomials and Expressions

To reuse Mathlib's results on ideals and varieties, polynomials are represented as `MvPolynomial Nat (ZMod p)`:

- This type is *abstract*: evaluation and root enumeration are not computable;

Representing Polynomials and Expressions

To reuse Mathlib's results on ideals and varieties, polynomials are represented as `MvPolynomial Nat (ZMod p)`:

- This type is *abstract*: evaluation and root enumeration are not computable;
- We introduce an intermediate computable AST, `ZMod.Expr p`, with `eval` (evaluation function) and `toPoly` (embedding into Mathlib's representation).

Representing Polynomials and Expressions

To reuse Mathlib's results on ideals and varieties, polynomials are represented as `MvPolynomial Nat (ZMod p)`:

- This type is *abstract*: evaluation and root enumeration are not computable;
- We introduce an intermediate computable AST, `ZMod.Expr p`, with `eval` (evaluation function) and `toPoly` (embedding into Mathlib's representation).

Justifying `ff_root_branch` requires deciding that "`Roots`(f) is exhaustive": evaluate the expression over every field element.

Proof Rules $\longleftrightarrow$ Lean Lemmas

Each calculus rule is backed by a Lean lemma. E.g., `ff_ideal_generator`:

```
lemma ideal_generator {xs y zs : MvPolynomial Nat (ZMod p)}
    : y ∈ ideal (xs ++ y :: zs) := by exact Ideal.subset_span (by simp)
```

Proof Rules $\longleftrightarrow$ Lean Lemmas

Each calculus rule is backed by a Lean lemma. E.g., `ff_ideal_generator`:

```
lemma ideal_generator {xs y zs : MvPolynomial Nat (ZMod p)}
    : y ∈ ideal (xs ++ y :: zs) := by exact Ideal.subset_span (by simp)
```

For `ff_poly_norm` we implement a *verified* normalization procedure, `ZMod.Expr.toPolynomial`:

```
lemma eval_eq_of_beq {e₁ e₂ : Expr o}
    : e₁.toPolynomial == e₂.toPolynomial → e₁.eval ctx = e₂.eval ctx
```

Evaluation

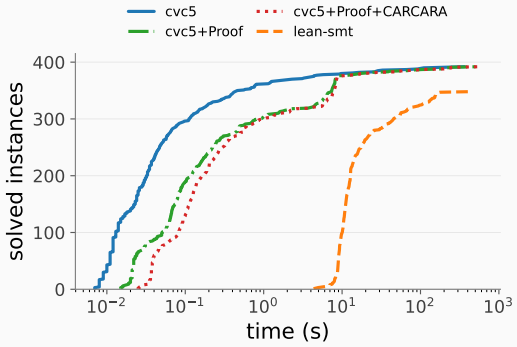
Evaluation - Setup

- Benchmarks: the 392 QF_FF instances from [**Ozdemir et al.**, **2023**] that cvc5 reports unsat within 20 min / 16GB, coming from various zero knowledge proof compilers;
- Configurations:
  - cvc5: baseline without proofs;
  - cvc5 `+Proof`: proof production in ALETHE;
  - cvc5 `+Proof+Carcara`: production and checking (CARCARA + FFPACHECK);
  - LEAN-SMT: full reconstruction inside Lean.

Evaluation - Results

Tool	Solved	Cum. time
cvc5	392	249s (1.0×)
cvc5 +Proof	392	600s (2.4×)
+CARCARA	392	642s (2.6×)
LEAN-SMT	348	9280s (37×)

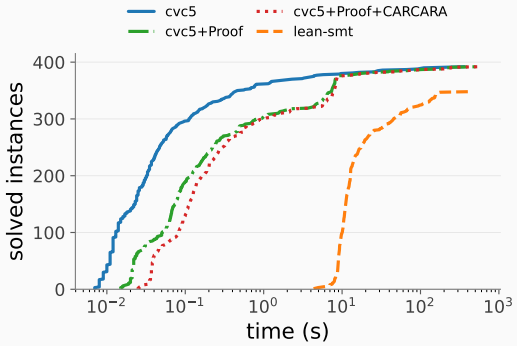
Cumulative times over the 348 commonly solved instances.



Evaluation - Results

Tool	Solved	Cum. time
cvc5	392	249s (1.0×)
cvc5 +Proof	392	600s (2.4×)
+CARCARA	392	642s (2.6×)
LEAN-SMT	348	9280s (37×)

Cumulative times over the 348 commonly solved instances.



- Producing *and checking* proofs solves the same 392 instances; checking adds only 0.50s on average, with ∼77% of the verification time spent in FFPACHECK;
- LEAN-SMT reconstructs 348 instances; runtimes are dominated by reconstruction and kernel type checking, most notably normalization for `ff_poly_norm`.

Conclusion

First proof production and checking pipeline for the SMT theory of finite fields:

- A proof calculus capturing cvc5's decision procedure, emitted as ALETHE and CPC certificates;
- FFPACHECK + CARCARA: efficient, independent checking, with overhead on par with other theories;
- Lean mechanization + LEAN-SMT: SMT automation in Lean 4 for finite field goals.

Conclusion

First proof production and checking pipeline for the SMT theory of finite fields:

- A proof calculus capturing cvc5's decision procedure, emitted as ALETHE and CPC certificates;
- FFPACHECK + CARCARA: efficient, independent checking, with overhead on par with other theories;
- Lean mechanization + LEAN-SMT: SMT automation in Lean 4 for finite field goals.

Future work:

- Extend the calculus to the *split* Gröbner basis procedure;
- Make Yices2's zero-decomposition-based procedure proof-producing.

References i

[**Abbott & Bigatti**] CoCoALib: A C++ Library for Doing Computations in Commutative Algebra. (tool)

[**Andreotti et al., 2023**] Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. (TACAS)

[**Barbosa et al., 2022**] cvc5: A Versatile and Industrial-Strength SMT Solver. (TACAS)

[**Kaufmann et al., 2024**] Practical Algebraic Calculus and Nullstellensatz with the Checkers Pacheck and Pastèque and Nuss-Checker. (FMSD)

[**Mohamed et al., 2025**] lean-SMT: An SMT Tactic for Discharging Proof Goals in Lean. (arXiv)

[**Ozdemir et al., 2023**] Satisfiability Modulo Finite Fields. (CAV)

References ii

[**Reynolds et al.**, **2026**] The Cooperating Proof Calculus: Comprehensive Proofs for an SMT Solver. (CAV, to appear)

[**Schurr et al.**, **2021**] Alethe: Towards a Generic SMT Proof Format. (PxTP)

Backup - The Cubic, End to End

The complete proof of the cubic example, with $f := x^3 + 2x + 2$ and $\tilde{f} := (x(x-1)(x-2) \approx 1)$:

$$\dfrac{\dfrac{\tilde{f} \qquad \dfrac{- \quad | \quad \ldots}{\tilde{f} \approx (f \approx 0)}}{f \approx 0} \qquad \dfrac{- \quad | \quad f, (f)}{f \in \langle f \rangle} \qquad | \quad x, \emptyset}{\bot}$$

Backup - Why Not the `ring` Tactic?

- `ring` renormalizes recursively inside the kernel: expensive on the large terms typical of SMT preprocessing, and it fails to scale in our experiments;
- `toPolynomial` computes one canonical form per relevant top-level expression and decides equality by syntactic comparison.

Backup - Deciding Root Exhaustiveness

"$\texttt{Roots}(f)$ is exhaustive" is decided by evaluating the expression over every field element:

```
def Expr.completeRoots [NeZero n] (p : Expr n) (i : Nat) (rs : List (ZMod n)) : Bool :=
  p.isUnivariateOver i && ∀ r, p.eval (fun _ => r) = 0 ↔ r ∈ rs
```

Backup - Coarse-to-Fine Proof Elaboration

During solving, each ideal membership is recorded as a single coarse-grained `macro_ff_poly_combination` step:

$$\frac{r_1 \in \langle G \rangle,\ \ldots,\ r_k \in \langle G \rangle \quad | \quad m_1, \ldots, m_k, f}{f \in \langle G \rangle} \text{ if } f = \textstyle\sum_i m_i r_i$$

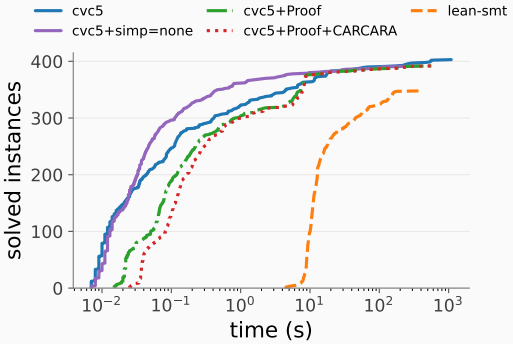
In a post-processing step, such applications can be refined into side condition free `ff_poly_combination` steps:

$$\dfrac{\dfrac{r_1 \in \langle G \rangle,\ \ldots,\ r_k \in \langle G \rangle}{\sum_{i=1}^{k} m_i \cdot r_i \in \langle G \rangle}\ \texttt{ff_poly_combination} \qquad \dfrac{-}{f \approx \sum_{i=1}^{k} m_i \cdot r_i}\ \texttt{ff_poly_norm}}{f \in \langle G \rangle}\ \text{congruence}$$

Backup - Full Evaluation

Tool	Solved	Cum. time
cvc5	403	
cvc5 +simp=none	392	249s (1.0×)
cvc5 +Proof	392	600s (2.4×)
+CARCARA	392	642s (2.6×)
LEAN-SMT	348	9280s (37×)

Cumulative times over the 348 commonly solved instances; slowdowns relative to cvc5 `+simp=none`.



Backup - `simplification=none` Option

By default, cvc5 runs a preprocessing pass that infers a substitution from the equalities in the input formula and applies it to eliminate symbols; `--simplification=none` disables it.

$\{\, y \approx x + 2,\quad y \cdot z \approx 1 \,\} \;\rightsquigarrow\; \text{substitute } y \mapsto x + 2 \;\rightsquigarrow\; \{\, (x + 2) \cdot z \approx 1 \,\}$ (y is eliminated)

- LEAN-SMT invokes cvc5 with this option natively, so cvc5 `+simp=none` is the baseline in the evaluation;
- Disabling the pass is what made us lose 11 instances ($403 \rightarrow 392$);